

Sound Classification Without Neural Networks

Pearl Bipin Pulickal

April 2025

Abstract

This paper presents a lightweight, traditional machine learning pipeline designed to classify phonetically similar spoken words — specifically, “Barbie” and “Puppy”. Despite a small initial dataset, the system achieves **92.37% accuracy** through a combination of handcrafted MFCC-based statistical features, custom audio augmentation, and optimized support vector machine (SVM) classification. The methodology is efficient, interpretable, and suitable for low-resource deployment.

1. Introduction

Spoken word classification becomes especially challenging when distinguishing phonetically similar terms. In this study, we focus on *Barbie* and *Puppy*, which sound closely related in many accents. Instead of relying on data-hungry deep learning techniques, this work explores a traditional machine learning pipeline with custom feature engineering and augmentation to achieve strong performance with limited resources.

2. Methodology

2.1 Dataset and Augmentation

The dataset included **98 audio samples** (50 “Barbie”, 48 “Puppy”) sourced from controlled recordings. Each audio clip was fixed to 3 seconds. To enhance generalization, five augmentations were applied per sample:

- Pitch shifting: +2 and -2 semitones
- Time stretching: 0.8× and 1.2×
- Gaussian noise

This expanded the dataset sixfold, resulting in **588 total samples**.

2.2 Feature Extraction

Each audio clip was transformed into 20 MFCCs. From each coefficient, we computed:

- **Unweighted stats:** mean, standard deviation, skewness, kurtosis, min, max
- **Weighted stats:** weighted mean, std, skew, kurtosis, min, max

This yielded **240-dimensional feature vectors** ($20 \text{ MFCCs} \times 12 \text{ stats}$).

2.3 Model Pipeline

The classification pipeline:

- Normalization with **StandardScaler**
- Dimensionality reduction via **PCA** (95% variance retained)
- Classification with **SVM (RBF kernel)** tuned using **GridSearchCV**

Best parameters found:

```
C = 10, gamma = 'scale', kernel = 'rbf'
```

Train-test split was 60-40 with stratification.

3. Results

- **Accuracy:** 92.37%
- **Precision:** 93% (Barbie), 92% (Puppy)
- **F1-Score:** 0.92 (overall)
- **Runtime:** 3 minutes

Confusion Matrix:

```
[[111   9]
 [  9 107]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.93	0.93	120
1	0.92	0.92	0.92	116

4. Discussion

The strong results validate the power of handcrafted MFCC features for distinguishing subtle acoustic differences. PCA helped reduce overfitting, while augmentation improved generalization. Traditional ML offers a fast, low-resource alternative to deep learning in this context.

5. Conclusion and Future Work

This work shows that thoughtful feature engineering and augmentation can yield high accuracy in spoken word classification tasks. Future directions include:

- Real-time inference on embedded/mobile devices
- Scaling to multi-class vocabularies
- Benchmarking against deep learning baselines (CNNs, wav2vec)

Appendix A: Code Screenshots

Resources ×

You are not subscribed. [Learn more](#)

You currently have zero compute units available. Resources offered free of charge are not guaranteed. Purchase more units [here](#).

At your current usage level, this runtime may last up to 55 hours 40 minutes.

[Manage sessions](#)

Want more memory and disk space? [Upgrade to Colab Pro](#) ×

Python 3 Google Compute Engine backend

Showing resources from 10:22 PM to 10:35 PM

System RAM
1.8 / 12.7 GB



Disk
37.1 / 107.7 GB



Code Screenshot 1

```
import os
import numpy as np
import librosa
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from google.colab import drive
from tqdm import tqdm
from scipy.stats import skew, kurtosis

# Mount Google Drive
drive.mount('/content/drive')
base_path = '/content/drive/MyDrive/BarbiePuppy/barbie_vs_puppy'
label_mapping = {'barbie': 0, 'puppy': 1}

# Define data augmentation function
def augment_audio(audio, sr):
    """
    Generate augmented versions of the audio.
    Returns a list of augmented audios.
    """
    augmented_audios = []

    # 1. Pitch shift by +2 semitones
    audio_pitch_up = librosa.effects.pitch_shift(audio, sr=sr, n_steps=2)
    augmented_audios.append(audio_pitch_up)

    # 2. Pitch shift by -2 semitones
    audio_pitch_down = librosa.effects.pitch_shift(audio, sr=sr, n_steps=-2)
    augmented_audios.append(audio_pitch_down)

    # 3. Time stretch: speed up (rate = 1.2)
    audio_fast = librosa.effects.time_stretch(audio, rate=1.2)
    # Fix length back to 3 seconds
    audio_fast = librosa.util.fix_length(audio_fast, size=3 * sr)
    augmented_audios.append(audio_fast)
```

Code Screenshot 2

```
# 4. Time stretch: slow down (rate = 0.8)
audio_slow = librosa.effects.time_stretch(audio, rate=0.8)
audio_slow = librosa.util.fix_length(audio_slow, size=3 * sr)
augmented_audios.append(audio_slow)

# 5. Add Gaussian noise
noise = 0.005 * np.std(audio) * np.random.randn(len(audio))
audio_noise = audio + noise
audio_noise = librosa.util.fix_length(audio_noise, size=3 * sr)
augmented_audios.append(audio_noise)

return augmented_audios

# Feature extraction function
def extract_features(audio, sr):
    """Computes both unweighted and weighted statistical features on MFCC coefficients."""
    mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=20)
    features = []

    for coeff in mfcc:
        # Unweighted statistics
        mean_val = np.mean(coeff)
        std_val = np.std(coeff)
        skew_val = skew(coeff)
        kurt_val = kurtosis(coeff)
        min_val = np.min(coeff)
        max_val = np.max(coeff)

        # Append unweighted features
        features.extend([mean_val, std_val, skew_val, kurt_val, min_val, max_val])

    # Define weights: using absolute value as weight
    weights = np.abs(coeff)
    total_weight = np.sum(weights)
```

Code Screenshot 3

```
# Avoid division by zero
if total_weight == 0:
    w_mean = 0
    w_variance = 0
else:
    # Weighted mean
    w_mean = np.sum(weights * coeff) / total_weight
    # Weighted variance
    w_variance = np.sum(weights * (coeff - w_mean) ** 2) / total_weight

w_std = np.sqrt(w_variance)

# Weighted skewness
if w_std != 0:
    w_skew = np.sum(weights * (coeff - w_mean) ** 3) / total_weight / (w_std ** 3)
else:
    w_skew = 0

# Weighted kurtosis
if w_std != 0:
    w_kurt = np.sum(weights * (coeff - w_mean) ** 4) / total_weight / (w_std ** 4)
else:
    w_kurt = 0

# Weighted min and max
w_min = np.min(weights * coeff)
w_max = np.max(weights * coeff)

# Append weighted features
features.extend([w_mean, w_std, w_skew, w_kurt, w_min, w_max])

return np.array(features)

# Load, process, and augment audio files
features = []
labels = []
for label_name, label_id in label_mapping.items():
    folder_path = os.path.join(base_path, label_name)
    for file_name in tqdm(os.listdir(folder_path), desc=f"Processing {label_name}"):
        if file_name.lower().endswith('.wav'):
            file_path = os.path.join(folder_path, file_name)
            # Load the audio file
            audio, sr = librosa.load(file_path, sr=22050)
```

Code Screenshot 4

```
# Load, process, and augment audio files
features = []
labels = []
for label_name, label_id in label_mapping.items():
    folder_path = os.path.join(base_path, label_name)
    for file_name in tqdm(os.listdir(folder_path), desc=f"Processing {label_name}"):
        if file_name.lower().endswith('.wav'):
            file_path = os.path.join(folder_path, file_name)
            # Load the audio file
            audio, sr = librosa.load(file_path, sr=22050)
            # Fix length to 3 seconds
            audio = librosa.util.fix_length(audio, size=3 * sr)

            # Add original sample
            features.append(extract_features(audio, sr))
            labels.append(label_id)

            # Generate and add augmented samples
            aug_audios = augment_audio(audio, sr)
            for aug_audio in aug_audios:
                features.append(extract_features(aug_audio, sr))
                labels.append(label_id)

# Convert lists to numpy arrays
features = np.array(features)
labels = np.array(labels)

# Train-test split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.4, random_state=42, stratify=labels
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Dimensionality reduction with PCA (retain 95% of variance)
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

Code Screenshot 5


```
labels = np.array(labels)

# Train-test split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.4, random_state=42, stratify=labels
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Dimensionality reduction with PCA (retain 95% of variance)
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Hyperparameter tuning with GridSearchCV for SVM
param_grid = {
    'C': [0.1, 1, 10], # Regularization parameter
    'gamma': ['scale', 'auto', 0.1], # Kernel coefficient
    'kernel': ['rbf', 'linear', 'poly'] # Types of kernels to try
}

grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_pca, y_train)

# Output best parameters
print(f"Best parameters found: {grid_search.best_params_}")

# Train the SVM with the best found parameters
svm_best = grid_search.best_estimator_

# Make predictions and evaluate the model
y_pred = svm_best.predict(X_test_pca)
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Code Screenshot 6

Appendix B: Performance Output Screenshot



Mounted at /content/drive

Processing barbie: 100% 50/50 [01:07<00:00, 1.35s/it]

Processing puppy: 100% 48/48 [00:54<00:00, 1.14s/it]

Best parameters found: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}

Model Accuracy: 92.37%

Confusion Matrix:

[[111 9]

[9 107]]

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.93	0.93	120
1	0.92	0.92	0.92	116
accuracy			0.92	236
macro avg	0.92	0.92	0.92	236
weighted avg	0.92	0.92	0.92	236

✓ 3m 13s

completed at 10:25PM

Performance Metrics Output Screenshot